

DATA PREPARATION

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [2]: movies = pd.read_csv(r"C:\Users\DEII\OneDrive\Dokumente\Desktop\HNG13 Internship\movies.csv")
movies.head(10)
```

```
Out[2]:
```

	movieId	title	genres
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	2	Jumanji (1995)	Adventure Children Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama Romance
4	5	Father of the Bride Part II (1995)	Comedy
5	6	Heat (1995)	Action Crime Thriller
6	7	Sabrina (1995)	Comedy Romance
7	8	Tom and Huck (1995)	Adventure Children
8	9	Sudden Death (1995)	Action
9	10	GoldenEye (1995)	Action Adventure Thriller

```
In [3]: ratings = pd.read_csv(r"C:\Users\DEII\OneDrive\Dokumente\Desktop\HNG13 Internship\ratings.csv")
ratings.head(10)
```

```
Out[3]:
```

	userId	movieId	rating	timestamp
0	1	1	4.0	964982703
1	1	3	4.0	964981247
2	1	6	4.0	964982224
3	1	47	5.0	964983815
4	1	50	5.0	964982931
5	1	70	3.0	964982400
6	1	101	5.0	964980868
7	1	110	4.0	964982176
8	1	151	5.0	964984041
9	1	157	5.0	964984100

```
In [4]: tags = pd.read_csv(r"C:\Users\DEII\OneDrive\Dokumente\Desktop\HNG13 Internship\tags.csv")
tags.head()
```

Out[4]:

	userId	movieId	tag	timestamp
0	2	60756	funny	1445714994
1	2	60756	Highly quotable	1445714996
2	2	60756	will ferrell	1445714992
3	2	89774	Boxing story	1445715207
4	2	89774	MMA	1445715200

In [5]: `links = pd.read_csv(r"C:\Users\DEII\OneDrive\Dokumente\Desktop\HNG13 Internship\links.csv")`
`links.head()`

Out[5]:

	movieId	imdbId	tmdbId
0	1	114709	862.0
1	2	113497	8844.0
2	3	113228	15602.0
3	4	114885	31357.0
4	5	113041	11862.0

In [6]: `df = ratings.merge(movies, on='movieId', how='left')`
`tags_grouped = tags.groupby('movieId')['tag'].apply(lambda x: ', '.join(x)).reset_index()`
`df = df.merge(links, on='movieId', how='left')`
`df = df.merge(tags_grouped, on='movieId', how='left')`
`df.head`

```
Out[6]: <bound method NDFrame.head of
title \
0      1      1      4.0    964982703      Toy Story (1995)
1      1      3      4.0    964981247      Grumpier Old Men (1995)
2      1      6      4.0    964982224      Heat (1995)
3      1     47      5.0    964983815      Seven (a.k.a. Se7en) (1995)
4      1     50      5.0    964982931      Usual Suspects, The (1995)
...      ...      ...      ...      ...
100831    610    166534    4.0    1493848402      Split (2017)
100832    610    168248    5.0    1493850091      John Wick: Chapter Two (2017)
100833    610    168250    5.0    1494273047      Get Out (2017)
100834    610    168252    5.0    1493846352      Logan (2017)
100835    610    170875    3.0    1493846415      The Fate of the Furious (2017)

genres      imdbId      tmdbId \
0      Adventure|Animation|Children|Comedy|Fantasy      114709      862.0
1      Comedy|Romance      113228      15602.0
2      Action|Crime|Thriller      113277      949.0
3      Mystery|Thriller      114369      807.0
4      Crime|Mystery|Thriller      114814      629.0
...      ...      ...
100831      Drama|Horror|Thriller      4972582      381288.0
100832      Action|Crime|Thriller      4425200      324552.0
100833      Horror      5052448      419430.0
100834      Action|Sci-Fi      3315342      263115.0
100835      Action|Crime|Drama|Thriller      4630562      337339.0

tag
0      pixar, pixar, fun
1      moldy, old
2      NaN
3      mystery, twist ending, serial killer
4      mindfuck, suspense, thriller, tricky, twist en...
...      ...
100831      NaN
100832      action, dark hero, gun tactics, hitman, Keanu ...
100833      NaN
100834      dark, emotional, gritty, heartbreaking, predic...
100835      NaN

[100836 rows x 9 columns]>
```

```
In [7]: df
```

Out[7]:

	userId	movieId	rating	timestamp	title	genres	imdbId
0	1	1	4.0	964982703	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	1
1	1	3	4.0	964981247	Grumpier Old Men (1995)	Comedy Romance	1
2	1	6	4.0	964982224	Heat (1995)	Action Crime Thriller	1
3	1	47	5.0	964983815	Seven (a.k.a. Se7en) (1995)	Mystery Thriller	1
4	1	50	5.0	964982931	Usual Suspects, The (1995)	Crime Mystery Thriller	1
...	...	...	...	...	...	...	...
100831	610	166534	4.0	1493848402	Split (2017)	Drama Horror Thriller	49
100832	610	168248	5.0	1493850091	John Wick: Chapter Two (2017)	Action Crime Thriller	44
100833	610	168250	5.0	1494273047	Get Out (2017)	Horror	50
100834	610	168252	5.0	1493846352	Logan (2017)	Action Sci-Fi	33
100835	610	170875	3.0	1493846415	The Fate of the Furious (2017)	Action Crime Drama Thriller	46

100836 rows × 9 columns



```
In [8]: df.isnull().sum()
```

```
Out[8]: userId      0
movieId      0
rating       0
timestamp    0
title        0
genres       0
imdbId       0
tmdbId       13
tag        52549
dtype: int64
```

```
In [9]: df.duplicated().sum()
```

Out[9]: 0

```
In [10]: df.drop_duplicates(inplace=True)
```

```
In [11]: df['tag'] = df['tag'].fillna('No Tag')
```

```
In [12]: df.dropna(subset=['tmdbId'], inplace=True)
print(df.isnull().sum())
```

```
userId      0
movieId     0
rating      0
timestamp   0
title       0
genres      0
imdbId      0
tmdbId      0
tag         0
dtype: int64
```

```
In [13]: df['timestamp'] = pd.to_datetime(df['timestamp'], unit='s')
df['year'] = df['timestamp'].dt.year
df['month'] = df['timestamp'].dt.month
```

FEATURES ENGINEERING

```
In [14]: # 1. Extract release year from title
df['release_year'] = df['title'].str.extract(r'\((\d{4})\)').astype(float)
```

```
In [15]: # 2. Number of genres
df['num_genres'] = df['genres'].apply(lambda x: len(x.split('|')))
```

```
In [16]: # 3. Average rating per movie
movie_avg = df.groupby('movieId')['rating'].mean().reset_index(name='avg_rating').round(2)
df = df.merge(movie_avg, on='movieId', how='left')
```

```
In [17]: # 4: User Rating Frequency - Count of ratings per user
user_freq = df.groupby('userId')['rating'].count().reset_index(name='user_rating_freq')
df = pd.merge(df, user_freq, on='userId', how='left')
```

```
In [18]: # 5: Movie Age - Years since release (using current date: Oct 20, 2025)
current_year = 2025
df['movie_age'] = current_year - df['release_year']
```

```
In [19]: # Feature 6: Number of Tags per Movie - Aggregate tags
num_tags = tags.groupby('movieId')['tag'].count().reset_index(name='num_tags')
df = pd.merge(df, num_tags, on='movieId', how='left')
df['num_tags'] = df['num_tags'].fillna(0)
```

```
In [21]: df.head()
```

Out[21]:

	userId	movieId	rating	timestamp	title	genres	imdbId
0	1	1	4.0	2000-07-30 18:45:03	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	114709
1	1	3	4.0	2000-07-30 18:20:47	Grumpier Old Men (1995)	Comedy Romance	113228
2	1	6	4.0	2000-07-30 18:37:04	Heat (1995)	Action Crime Thriller	113277
3	1	47	5.0	2000-07-30 19:03:35	Seven (a.k.a. Se7en) (1995)	Mystery Thriller	114369
4	1	50	5.0	2000-07-30 18:48:51	Usual Suspects, The (1995)	Crime Mystery Thriller	114814

In [40]:

```
df = df[
    ['userId', 'movieId', 'title', 'rating', 'timestamp', 'genres', 'imdbId', 'tmdbId',
     'tag', 'avg_rating', 'movie_age', 'age_group', 'num_genres',
     'num_tags', 'year', 'month', 'release_year', 'user_rating_freq']]
```

In [41]:

```
df.head()
```

Out[41]:

	userId	movieId	title	rating	timestamp	genres	imdbId
0	1	1	Toy Story (1995)	4.0	2000-07-30 18:45:03	Adventure Animation Children Comedy Fantasy	114709
1	1	3	Grumpier Old Men (1995)	4.0	2000-07-30 18:20:47	Comedy Romance	113228
2	1	6	Heat (1995)	4.0	2000-07-30 18:37:04	Action Crime Thriller	113277
3	1	47	Seven (a.k.a. Se7en) (1995)	5.0	2000-07-30 19:03:35	Mystery Thriller	114369
4	1	50	Usual Suspects, The (1995)	5.0	2000-07-30 18:48:51	Crime Mystery Thriller	114814

In [104...]

```
df.to_csv("cleaned_movielens.csv", index=False, sep=',')
```

EXPLORATORY DATA ANALYSIS

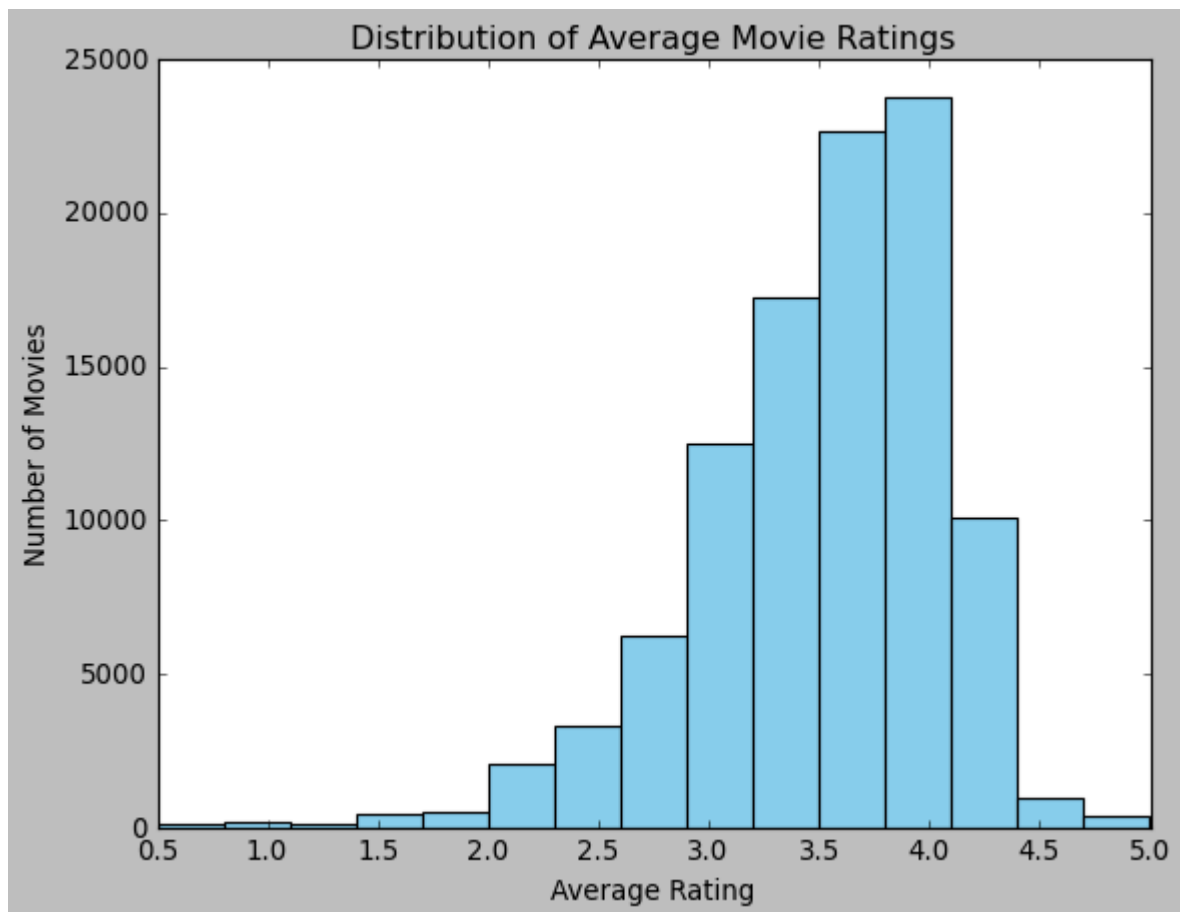
In [86]:

```
print(plt.style.available)
plt.style.use('classic')
```

```
%matplotlib inline
```

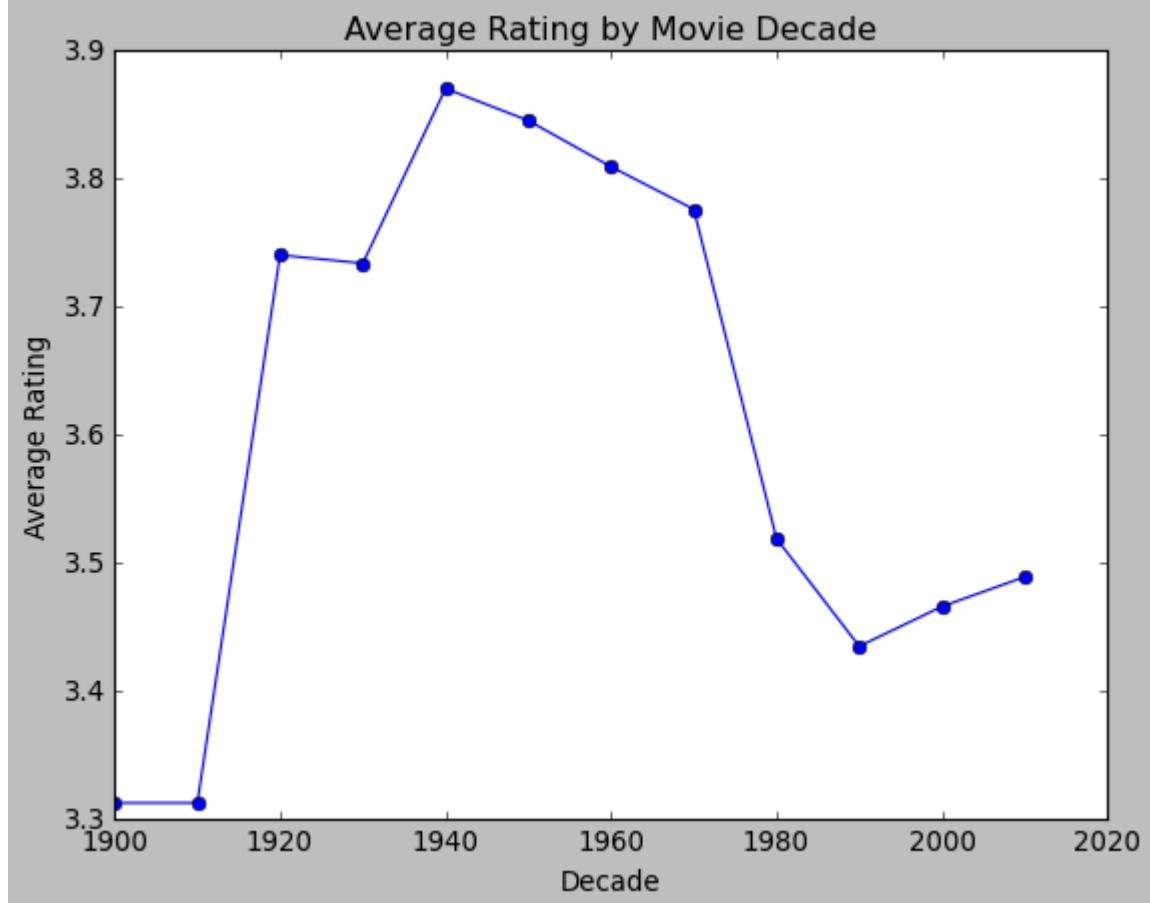
```
['Solarize_Light2', '_classic_test_patch', '_mpl-gallery', '_mpl-gallery-nogrid', 'bmh', 'classic', 'dark_background', 'fast', 'fivethirtyeight', 'ggplot', 'grayscale', 'seaborn-v0_8', 'seaborn-v0_8-bright', 'seaborn-v0_8-colorblind', 'seaborn-v0_8-dark', 'seaborn-v0_8-dark-palette', 'seaborn-v0_8-darkgrid', 'seaborn-v0_8-deep', 'seaborn-v0_8-muted', 'seaborn-v0_8-notebook', 'seaborn-v0_8-paper', 'seaborn-v0_8-pastel', 'seaborn-v0_8-poster', 'seaborn-v0_8-talk', 'seaborn-v0_8-ticks', 'seaborn-v0_8-white', 'seaborn-v0_8-whitegrid', 'tableau-colorblind10']
```

```
In [99]: plt.hist(df['avg_rating'], bins=15, color='skyblue', edgecolor='black')
plt.title('Distribution of Average Movie Ratings')
plt.xlabel('Average Rating')
plt.ylabel('Number of Movies')
plt.savefig("Distribution_of_Average_Movie_Ratings.png", dpi=300, bbox_inches='tight')
plt.show()
```



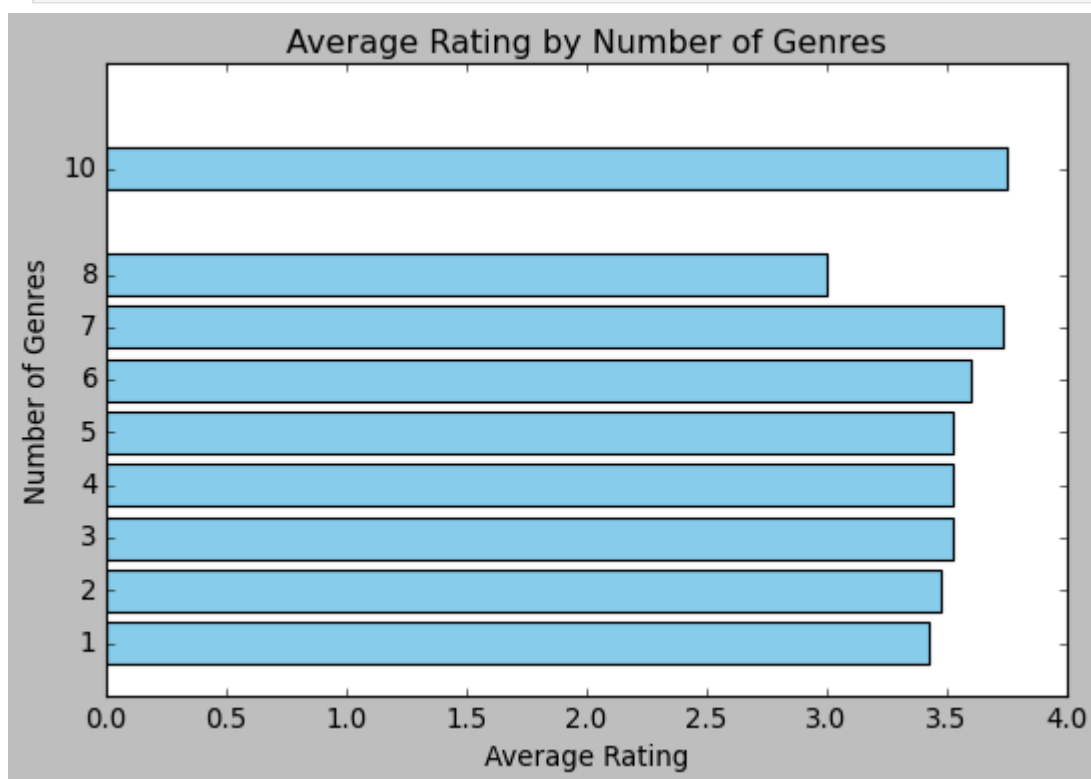
```
In [98]:
```

```
In [100... df['decade'] = (df['release_year'] // 10) * 10
rating_by_decade = df.groupby('decade')['avg_rating'].mean().reset_index()
plt.plot(rating_by_decade['decade'], rating_by_decade['avg_rating'], marker='o')
plt.title('Average Rating by Movie Decade')
plt.xlabel('Decade')
plt.ylabel('Average Rating')
plt.savefig("Average_Rating_by_Movie_Decade.png", dpi=300, bbox_inches='tight')
plt.show()
```



In [101...

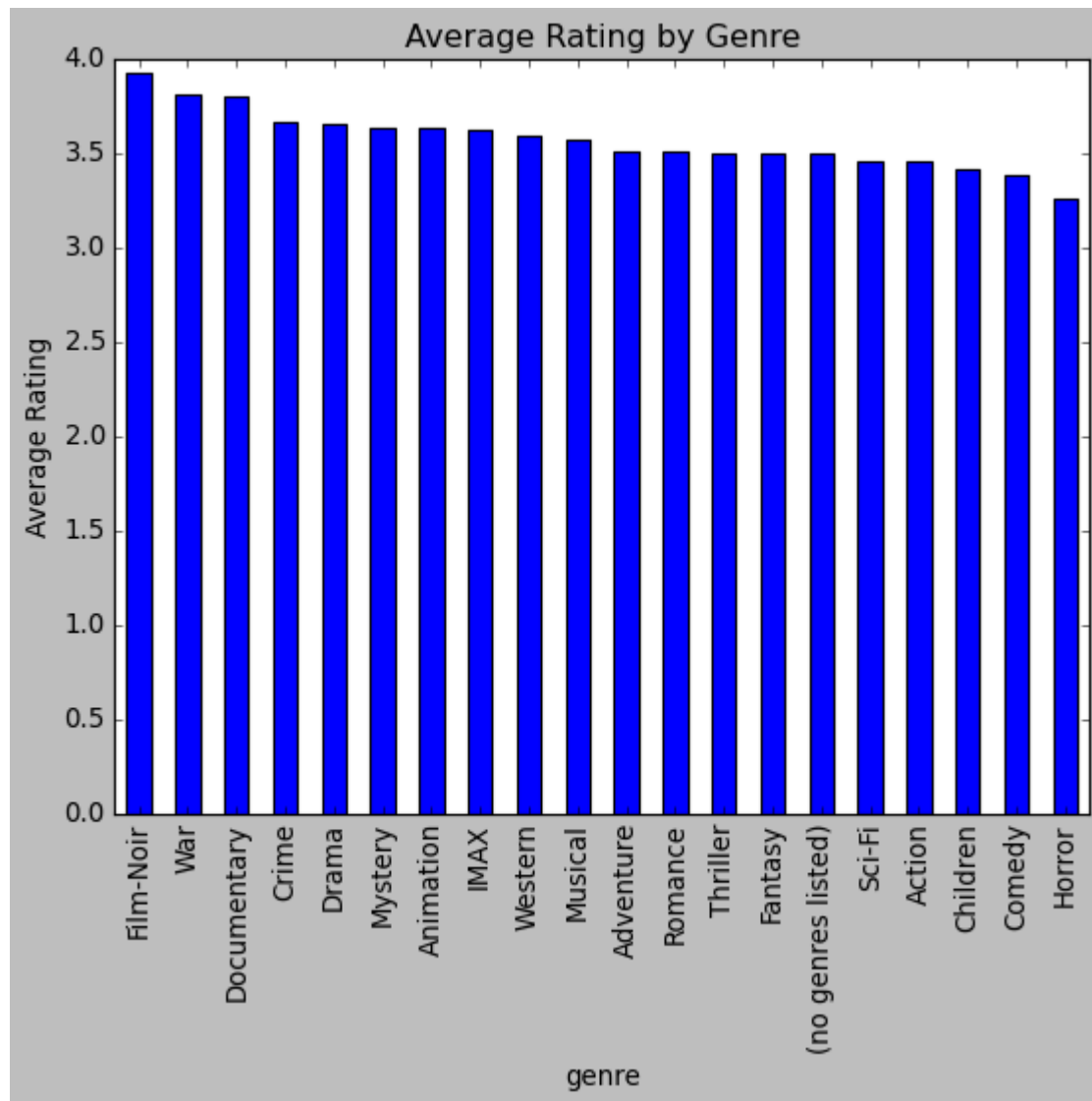
```
genre_rating = df.groupby('num_genres')['avg_rating'].mean().reset_index()
plt.figure(figsize=(7,5))
plt.barh(genre_rating['num_genres'], genre_rating['avg_rating'], color='skyblue', edgecolor='black')
plt.yticks(genre_rating['num_genres']) # ensures 1,2,3... appear
plt.title('Average Rating by Number of Genres')
plt.xlabel('Average Rating')
plt.ylabel('Number of Genres')
plt.tight_layout()
plt.savefig("Average_Rating_by_Number_of_Genres.png", dpi=300, bbox_inches='tight')
plt.show()
```



In [102...

```
genre_df = df.assign(genre=df['genres'].str.split('|')).explode('genre')
genre_rating = genre_df.groupby('genre')['avg_rating'].mean().sort_values(ascending=False)
genre_rating.plot(kind='bar')
```

```
plt.title('Average Rating by Genre')
plt.ylabel('Average Rating')
plt.savefig("Average_Rating_by_Genres.png", dpi=300, bbox_inches='tight')
plt.show()
```

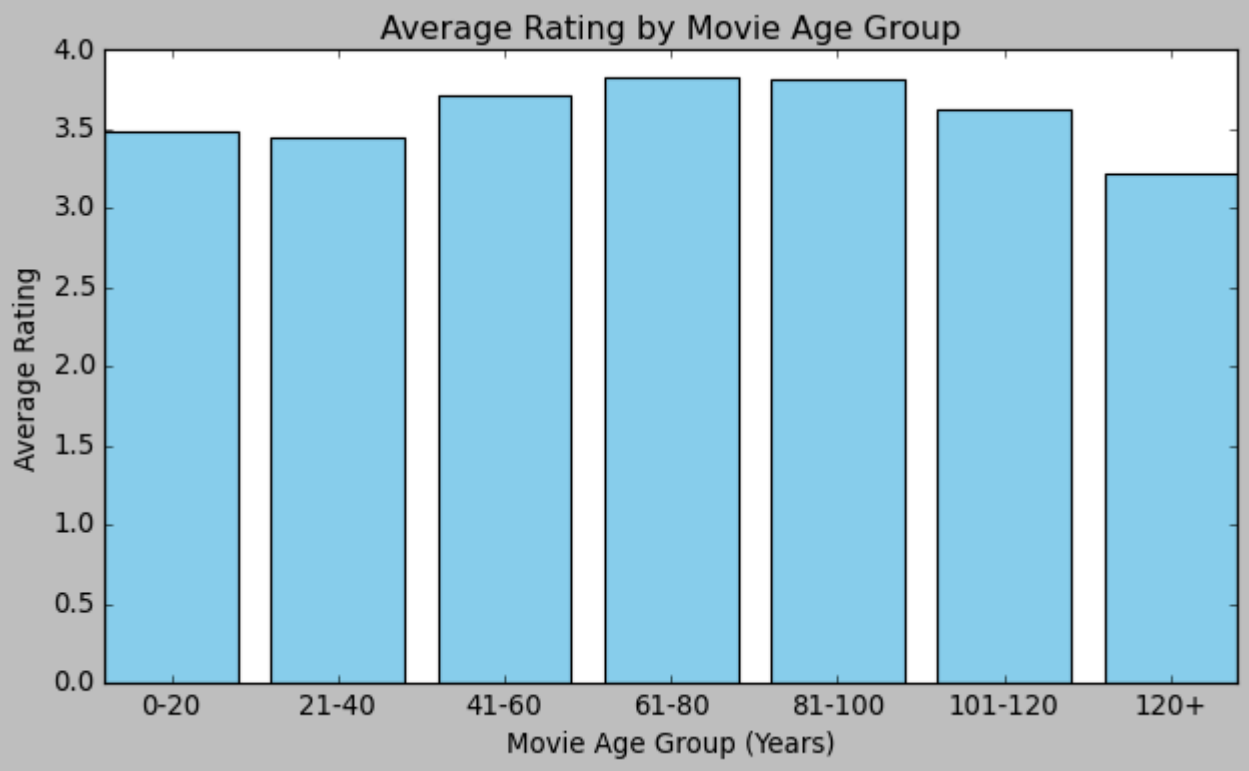


In [103...

```
df['age_group'] = pd.cut(
    df['movie_age'],
    bins=[0, 20, 40, 60, 80, 100, 120, float('inf')], # Added 120 and above
    labels=['0-20', '21-40', '41-60', '61-80', '81-100', '101-120', '120+']
)

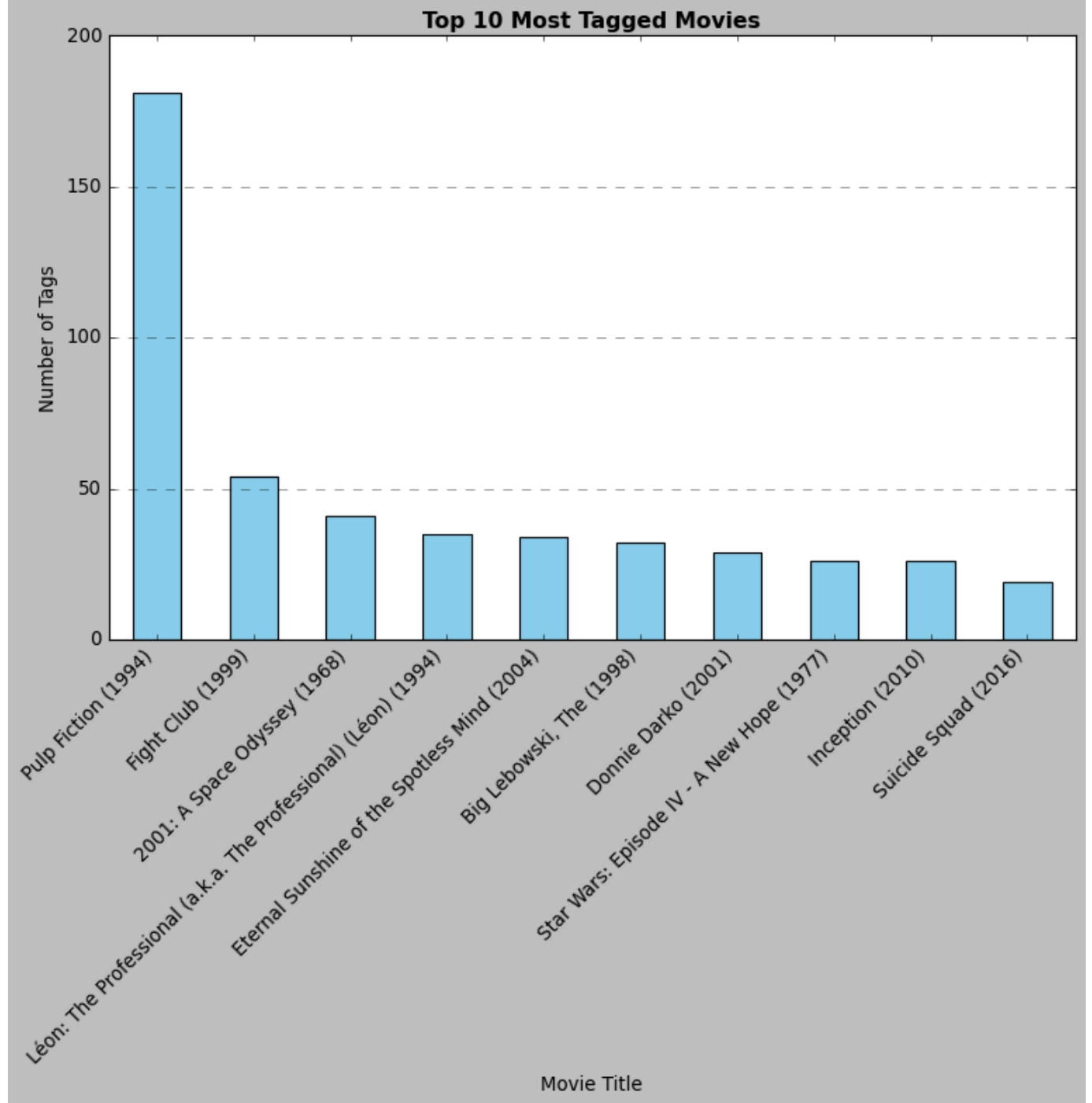
# Calculate average rating per age group
age_group_rating = df.groupby('age_group', observed=False)['avg_rating'].mean().reset_index()

plt.figure(figsize=(8,5))
plt.bar(age_group_rating['age_group'], age_group_rating['avg_rating'], color='skyblue', edgecolor='black')
plt.title('Average Rating by Movie Age Group')
plt.xlabel('Movie Age Group (Years)')
plt.ylabel('Average Rating')
plt.tight_layout()
plt.savefig("Average_Rating_by_Movie_Age_Group.png", dpi=300, bbox_inches='tight')
plt.show()
```



In [114...

```
top_tagged = df.groupby('title')['num_tags'].max().sort_values(ascending=False).head(10)
plt.figure(figsize=(10, 10))
top_tagged.plot(kind='bar', color='skyblue', edgecolor='black')
plt.title('Top 10 Most Tagged Movies', fontsize=14, fontweight='bold')
plt.xlabel('Movie Title', fontsize=12)
plt.ylabel('Number of Tags', fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.savefig("Top_10_Most_Tagged_Movies.png", dpi=300, bbox_inches='tight')
plt.show()
```



In []: